Duoc UC
Sede Maipú · Ingeniería en Informática

Informe de Avance del Proyecto APT

RID — RepInDepth

Aplicación móvil de entrenamiento de fuerza con motor probabilístico de prescripción

Asignatura	Capstone (PTY4614)
Evaluación	Fase 2, semana 9
Proyecto	APT122
Fecha	15 de octubre de 2026

Integrantes

Fernando Almonacid — Arquitectura y backend
Bastián Berríos — Frontend móvil y UX
Favio Pérez — DevOps, base de datos y QA

Plan 1446114 · Sección 005D

Índice general

Abstract **2**

1 Resumen de avance y metodología **2**

1.1 Actividades ejecutadas y objetivos cumplidos 2

1.2 Hitos del cronograma 3

1.3 Metodología aplicada 3

2 Ajustes al proyecto **4**

3 Evidencias de avance disciplinar **5**

3.1 Limitaciones de la evidencia 6

4 Modelo de datos y modelo de la aplicación **6**

4.1 Modelo de datos del servidor 6

4.2 Modelo actual de la aplicación 8

5 Monitoreo del plan de trabajo **10**

5.1 Factores que facilitaron el desarrollo 11

5.2 Factores que dificultaron el desarrollo 12

5.3 Plan de acción 12

Referencias **13**

Anexo de evidencias **13**

Abstract

Español

El Proyecto RID (RepInDepth) desarrolla una aplicación móvil de entrenamiento de fuerza con arquitectura *local-first*, cuyo motor probabilístico prescribe carga, repeticiones y semanas de descarga a partir de series registradas por voz. El dispositivo captura, sincroniza y consulta; el servidor prescribe. Este informe da cuenta del avance al 15 de octubre de 2026, dentro de un cronograma de 16 semanas (13 de agosto al 27 de noviembre) conducido con una metodología híbrida basada en RUP, con iteraciones semanales. A la fecha, el backend está desplegado y verificado (migraciones 001 a 015, 51 casos de pruebas de seguridad a nivel de fila y 266 pruebas automatizadas del motor), se diseñaron 68 pantallas y se construyó el cliente móvil con Vite, Preact y Capacitor, validado en emulador con inicio de sesión, lista de sesiones, plan prescrito por la función de borde y consulta de progreso. Permanecen pendientes el registro de series, el reconocimiento por voz, la operación sin conexión y el flujo de suscripción; estas actividades presentan retraso respecto del plan y se abordan con las acciones descritas en el apartado de monitoreo.

English

Project RID (RepInDepth) develops a local-first mobile strength-training application whose probabilistic engine prescribes load, repetitions and deload weeks from sets recorded by voice. The device captures, synchronizes and queries; the server prescribes. This report presents the progress made as of October 15, 2026, within a 16-week schedule (August 13 to November 27) managed with a hybrid RUP-based methodology and weekly iterations. To date, the backend has been deployed and verified (migrations 001 to 015, 51 row-level security test cases and 266 automated engine tests), 68 screens have been designed, and the mobile client has been built with Vite, Preact and Capacitor and validated on an emulator, covering sign-in, session listing, the plan prescribed by the edge function, and progress queries. Set logging, voice recognition, offline operation and the subscription flow remain pending; these activities are behind schedule and are addressed by the actions described in the monitoring section.

1 Resumen de avance y metodología

El motor y el backend están terminados y el cliente móvil ya consume datos reales, pero las capacidades de captura (registro de series, voz y operación sin conexión) siguen pendientes, por lo que el MVP no está aún integrado.

1.1 Actividades ejecutadas y objetivos cumplidos

- **Requisitos y arquitectura.** Se priorizaron los requisitos con MoSCoW (30 requisitos funcionales y 10 no funcionales) y se documentó la arquitectura en vistas 4+1, con diagramas UML, C4, entidad-relación, de secuencia y de estado.
- **Datos y seguridad.** El esquema PostgreSQL se encuentra desplegado en Supabase mediante las migraciones 001 a 015, con catálogo de 470 ejercicios y seguridad a nivel de fila (RLS) verificada

con 51 casos de prueba cruzados entre usuarios.

- **Motor de prescripción.** El paquete `@rid/engine` está operativo, cuenta con 266 pruebas automatizadas y se despliega como función de borde de Supabase.
- **Interfaz.** Se diseñaron 68 pantallas de teléfono y tablet en nueve iteraciones, con tokens de diseño y contrastes documentados (ADR-007).
- **Cliente móvil.** Se construyó la aplicación con Vite, Preact y Capacitor 8: autenticación completa (alta, verificación por código, recuperación y cambio de contraseña), lista de sesiones, plan prescrito por el servidor y consulta de progreso. El flujo se verificó en emulador el 14 de octubre de 2026.

1.2 Hitos del cronograma

Hito	Fecha planificada	Situación	Respaldo
H1 · Alcance y requisitos cerrados	21-ago	**Cumplido**	Requisitos priorizados y documentados
H2 · Arquitectura y modelo de datos validados	31-ago	**Cumplido**	Vistas 4+1, ADR y migraciones
H3 · Motor matemático v1 funcional	22-sep	**Cumplido**	Motor con 266 pruebas automatizadas
H4 · Capa de datos y sincronización operativa	25-sep	**Parcial**	Datos y RLS operativos; sincronización del cliente pendiente
H5 · MVP integrado e instalable	19-oct	**En riesgo**	Integración verificada en emulador; faltan registro de series, voz y operación sin conexión

1.3 Metodología aplicada

El proyecto sigue una metodología híbrida basada en RUP, con iteraciones semanales a lo largo de 16 semanas (13 de agosto al 27 de noviembre de 2026). Cuatro prácticas la hacen verificable:

1. **Decisión documentada antes del código.** Cada decisión mayor se registra como ADR con contexto, alternativas descartadas y consecuencias (ADR-007, ADR-008 y ADR-009 corresponden a esta etapa).
2. **Separación de responsabilidades.** El dispositivo captura, sincroniza y consulta; el servidor prescribe. La aplicación no incluye datos de muestra: si falta un contrato del servidor, la pantalla muestra un error real.
3. **Pruebas como criterio de cierre.** Se combinan pruebas adversariales de RLS, la suite del motor, pruebas unitarias del cliente y verificación en emulador.
4. **Seguimiento sistemático.** El avance se controla con un cronograma en Microsoft Project (24 actividades, 10 hitos y 3 fases) y un backlog de 40 tareas en Zoho Projects.

2 Ajustes al proyecto

Se realizaron siete ajustes respecto del plan inicial. Seis responden a dificultades técnicas o de producto detectadas durante la construcción, y todos se registraron como decisiones documentadas o como correcciones verificadas.

Ajuste	Motivo (dificultad, facilitador o retroalimentación)	Efecto sobre el plan
Identidad visual: de tema casi negro con acento menta y Space Grotesk a fondo morado con un único acento amarillo, tipografía de sistema y sin degradados (ADR-007, 3 de octubre; Almonacid, 2026a)	Nueva dirección de producto: transmitir avance y energía, seguir las Human Interface Guidelines de Apple y priorizar el funcionamiento sobre el ornamento. Los contrastes se calcularon según WCAG 2.1 (mínimo AA)	Se reemplazan los tokens antes de construir más componentes, se retira la fuente y se suspende el gradiente de esfuerzo; el RIR se comunica con número y etiqueta
Base del cliente móvil: de Astro a Vite + Preact con Capacitor 8; Astro se reserva para RID Web (ADR-008, 4 de octubre; Almonacid, 2026b)	Dificultad: la aplicación resultó en 68 pantallas con navegación continua, estado de entrenamiento y almacenamiento local, donde Astro solo añadía una capa. Facilitador: los prototipos HTML/CSS comparten los tokens y el motor está en TypeScript	Se reemplaza el esqueleto inicial (tarea C1). Se pospone Kotlin Multiplatform por el costo de reconstruir las pantallas y el plazo del 27 de noviembre
Reconocimiento de voz: dos vías posibles, el fork `@capgo/capacitor-speech-recognition` o un plugin propio en Kotlin sobre `SpeechRecognizer`	Dificultad: el plugin de la comunidad está en la versión 7 y no es compatible con Capacitor 8	La decisión se difiere a una prueba en dispositivo físico (tarea C4); constituye un riesgo para la actividad 21
Alcance de los prototipos: de wireframes a 68 pantallas de alta fidelidad en nueve iteraciones (actividad 6)	Facilitador: con el diseño cerrado se programa sin rediseñar y el traspaso a Preact es casi directo	La actividad 6, prevista hasta el 2 de septiembre, se extendió hasta la iteración 9 de octubre
Selección de la sesión del día: la persona elige la sesión; el dispositivo no decide cuál corresponde	Dificultad: el esquema no tiene programa ni calendario, y decidirlo en el cliente sería prescribir en el dispositivo, contra la regla de arquitectura	Se solicita al servidor una regla (por ejemplo, una función RPC `sesion_de_hoy`); mientras tanto, Hoy lista las sesiones
Recuperación de contraseña por código en lugar de enlace (ADR-009, propuesta del 13 de octubre; Equipo RID, 2026a)	Dificultad: un enlace exige App Links con dominio verificado, filtro de intents y redirección en Supabase. El código funciona dentro de la aplicación y es coherente con la verificación del alta	Cambia el texto del diseño («Enviar enlace» pasa a «Enviar código»); pendiente de aprobación
Integración con la función de borde: cabecera `x-client-info` en CORS y vista `v_diagnostico_ultimo` con `security_invoker`	Dificultad detectada al integrar: el WebView bloqueaba la llamada y la vista podía no respetar RLS del invocador	Corrección aplicada y verificada en emulador; falta incorporarla a una migración

3 Evidencias de avance disciplinar

El avance cuenta con evidencia en las cuatro competencias del perfil de egreso. La brecha principal está en el dispositivo: el registro de series, la voz y la operación sin conexión aún no tienen evidencia porque no están construidos. Las rutas de cada evidencia en el repositorio se detallan en el anexo.

Competencia · indicador de calidad	Evidencia	Resultado verificable
Pruebas · 1.1 Diseña pruebas de validación	Suite de seguridad a nivel de fila (RLS) con casos cruzados entre usuarios (A→B y B→A), suite del motor y prueba de humo (`npm run humo`)	51 casos RLS; 266 pruebas del motor en 10 archivos; prueba de humo con 34 aciertos y 0 fallos en 13 secciones
Pruebas · 1.2 Aplica pruebas de validación	Ejecución de las suites, 42 pruebas unitarias del cliente (estados de recursos, flujo de verificación por código, caché del plan, traducción de errores) y verificación en emulador	Flujo real verificado el 14 de octubre: inicio de sesión, lista de sesiones, plan desde la función de borde y progreso
Pruebas · 1.3 Desarrolla mejoras a partir de los resultados	Corrección de los hallazgos de las pruebas	10 fallos originales de RLS corregidos (vistas sin `security_invoker`, suplantación en triggers, columnas derivadas escribibles por el cliente, tablas sin RLS); CORS con `x-client-info`
Gestión · 2.1 Planifica	Cronograma en Microsoft Project con predecesoras e hitos, backlog de 40 tareas en Zoho Projects, requisitos priorizados con MoSCoW	24 actividades, 10 hitos y 3 fases; 30 requisitos funcionales y 10 no funcionales
Gestión · 2.2 Controla	Registro de decisiones con alternativas y consecuencias, y la tabla de monitoreo de este informe	Ocho ADR (001 a 003 y 005 a 009); ADR-008 compara cuatro alternativas, entre ellas Kotlin Multiplatform
Modelos de datos · 3.1 Diseña	Modelo entidad-relación y política de acceso por defecto (RLS automático y `auto_expose_new_tables = false`)	Esquema PostgreSQL con vistas diseñadas bajo RLS
Modelos de datos · 3.2 Implementa	Migraciones desplegadas en Supabase y catálogo de ejercicios	Migraciones 001 a 015; 470 ejercicios (16 anclas curadas y 454 enriquecidas automáticamente); 1.386 sustituciones
Software · 4.1 Construye	Motor `@rid/engine` (filtro de Kalman, modelo bayesiano jerárquico y predicción conformal) y cliente móvil por capas (infraestructura, rutas, datos, estado, componentes y pantallas)	Unas 1.600 líneas en el cliente; ninguna calcula cargas ni e1RM

Competencia · indicador de calidad	Evidencia	Resultado verificable
Software · 4.2 Integra componentes	Cliente conectado a la función de borde `engine` (`GET /plan`) con tipos espejo de `sealed-plan.ts`; un único cliente Supabase con clave anon bajo RLS	Plan prescrito por el servidor visible en la pantalla de sesión
Software · 4.3 Implanta	Función de borde desplegada con reprocesamiento nocturno (`pg_cron` y `pg_net`) y aplicación empaquetada con Capacitor para Android	Verificada en emulador; la instalación en dispositivo físico y la publicación en Google Play (actividad 29) están pendientes

3.1 Limitaciones de la evidencia

- El flujo de alta, verificación y recuperación de contraseña se probó con un cliente simulado; aún no se prueba contra un proyecto real de Supabase con servidor SMTP propio.
- El plan de la función de borde se guarda en memoria; falta el almacenamiento local para la operación sin conexión.
- Las cifras de las pantallas de diseño son datos de muestra del prototipo. La aplicación real no incluye datos de muestra y consume únicamente datos del servidor.

4 Modelo de datos y modelo de la aplicación

Esta sección documenta los dos modelos que sostienen la evidencia anterior: el modelo de datos del servidor y la arquitectura actual del cliente móvil. La regla que ordena a ambos es la misma: el dispositivo captura, sincroniza y consulta; el servidor prescribe.

4.1 Modelo de datos del servidor

El servidor es un proyecto Supabase con PostgreSQL (migraciones 001 a 015). El esquema `public` se obtuvo mediante el volcado de la base desplegada y contiene 17 tablas con 156 columnas, 22 claves foráneas y 29 restricciones CHECK, además de 7 vistas y 15 tipos enumerados. La Figura 1 presenta el modelo entidad-relación, agrupado según la función de cada tabla; el diccionario completo de columnas figura en el anexo.

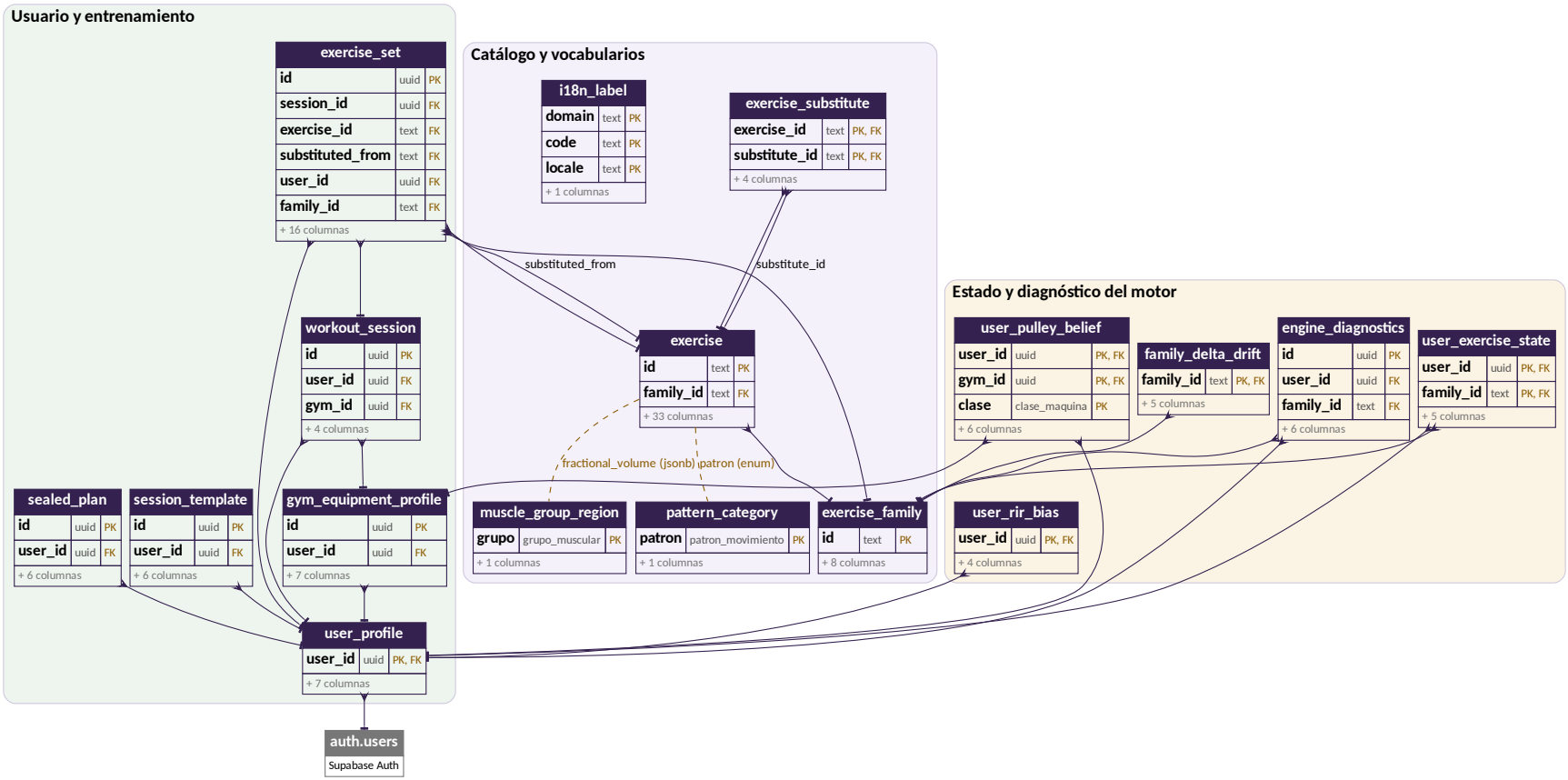


Figura 1. Modelo entidad-relación del esquema público (solo claves primarias y foráneas; las líneas punteadas indican vínculos lógicos mediante tipo enumerado o JSON, sin clave foránea).

Grupo	Tablas	Contenido y acceso del cliente
Catálogo y vocabularios	6	`exercise` (35 columnas), `exercise_family`, `exercise_substitute`, `muscle_group_region`, `pattern_category` e `i18n_label`. El usuario autenticado solo puede leerlas
Usuario y entrenamiento	6	`user_profile`, `gym_equipment_profile`, `session_template`, `sealed_plan`, `workout_session` y `exercise_set`. Contienen los datos propios del usuario
Estado y diagnóstico del motor	5	`user_exercise_state`, `user_rir_bias`, `user_pulley_belief`, `engine_diagnostics` y `family_delta_drift`. El cliente solo lee lo propio; las escribe el servidor con el rol de servicio, y la última carece de política a propósito

Las siguientes decisiones de diseño son visibles en el propio esquema y sostienen la regla de que el servidor prescribe:

- **El cliente no controla los datos derivados.** El trigger `rellenar_denormalizados` calcula `user_id` y `family_id` de cada serie a partir de la sesión y del ejercicio, y descarta el valor enviado. Además, el cliente solo tiene permisos de inserción y actualización sobre las columnas de captura de `exercise_set` (peso, repeticiones, RIR, lado, método de ingreso, confianza de voz, sustitución); el peso efectivo, el RIR corregido, el e1RM y su intervalo, `processed_at` y `modo_polea` los escribe solo el servidor.
- **Integridad del catálogo en la base.** Un ejercicio solo puede estar activo si está curado y tiene los parámetros que el motor necesita (`ck_active_requires_curation`), y solo es prescribible si está activo. El trigger `validate_fractional_volume` rechaza claves que no sean grupos musculares válidos y coeficientes fuera de (0, 1].
- **Aislamiento por usuario.** Las 19 políticas de seguridad a nivel de fila limitan cada tabla a las filas con `user_id = auth.uid()`; el catálogo y los vocabularios son de lectura para usuarios autenticados. La inserción de una serie exige que la sesión pertenezca al usuario, y una sesión con gimnasio exige que el gimnasio también lo sea.
- **Vistas.** Las tres vistas que consulta el cliente (`v_diagnostico_ultimo`, `v_session_balance` y `v_set_region_volume`) se crean con `security_invoker`, de modo que respetan las políticas de quien consulta; las otras cuatro son de administración y solo las lee el rol de servicio.
- **Borrado en cascada.** Todas las tablas de datos del usuario cuelgan de `user_profile`, que a su vez referencia `auth.users`, con `ON DELETE CASCADE`; eliminar la cuenta elimina sus datos.
- **Sin duplicados de series.** Una restricción de unicidad por sesión, ejercicio, índice de serie y lado impide registrar dos veces la misma serie.

4.2 Modelo actual de la aplicación

Arranque y estado de sesión. El estado de autenticación es una única señal (`auth`); la raíz de la aplicación decide qué mostrar según su valor, y ninguna pantalla navega por sí misma al entrar o salir.

Estado	Qué se muestra
`unconfigured`	Pantalla de configuración faltante (no hay variables `VITE_SUPABASE_*`)
`loading`	Pantalla vacía mientras se lee la sesión guardada
`signed-out`	Ingreso, crear cuenta, verificar código y recuperar contraseña
`recovery`	Contraseña nueva; no se permite entrar hasta guardarla
`signed-in`	Hoy, Progreso, Perfil y cambio de contraseña, con barra de pestañas

Capas. Cada capa solo conoce a las que tiene debajo, lo que mantiene fuera de la interfaz tanto el acceso al servidor como cualquier cálculo.

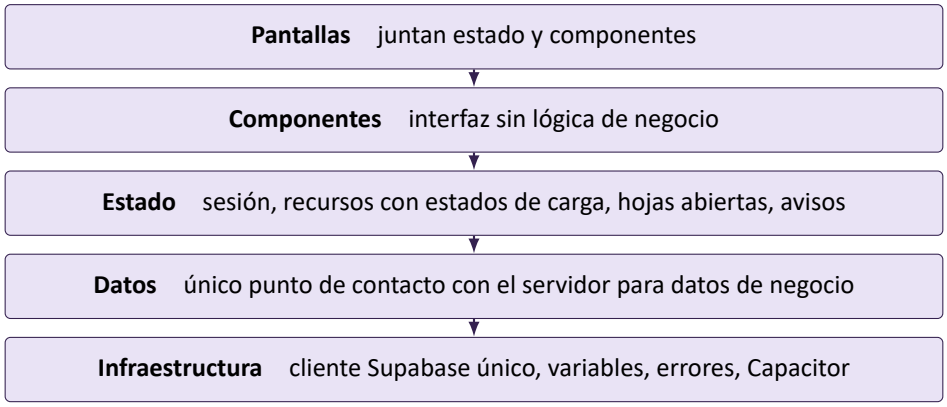


Figura 2. Capas del cliente móvil (de arriba hacia abajo: cada capa depende de la inferior). Estilos y rutas son transversales.

Cadena de las pantallas de datos.

- **Hoy:** al montarse lee las plantillas activas de `session_template` (bajo RLS) y las lista; al tocar una fila abre la sesión.
- **Sesión:** lee el nombre de la plantilla, pide el plan sellado a la función de borde (`GET /plan`, con caché por plantilla mientras el plan esté vigente), resuelve los nombres de ejercicio desde el catálogo y da formato al resultado sin calcular nada.
- **Progreso:** consulta `v_diagnostico_ultimo` filtrando por usuario y los nombres de familia; muestra la fuerza estimada que calculó el servidor, una fila por familia.

Diseño e implementación. El diseño comprende 68 pantallas, de las cuales el cliente tiene hoy conectadas las de cuenta (ingreso, alta, verificación por código, recuperación y cambio de contraseña) y las de datos (Hoy, Sesión, Progreso y Perfil).

Grupo del diseño	Pantallas	Contenido principal
Alta y acceso	16	Bienvenida, introducción, datos iniciales, cuenta, verificación, ingreso, suscripción y permisos
Núcleo diario	13	Hoy, sesión activa, sustitución, corrección de serie, resumen y récord, con estados de voz y sin conexión

Grupo del diseño	Pantallas	Contenido principal
Progreso y plan	10	Progreso, biblioteca, historial, plan, semana de descarga, bloque y objetivos
Perfil y cuenta	11	Datos, contraseña, suscripción, notificaciones, sincronización, unidades, exportación y eliminación de cuenta
Compartir e informes	11	Imágenes para compartir (Historia y cuadrado) e informes PDF
Tablet	5	Versiones de Hoy, sesión, progreso, resumen y compartir
Sistema	2	Error genérico y actualización requerida

Reglas que el código ya cumple. El servidor prescribe: ningún archivo del cliente calcula cargas, e1RM ni plan, y las validaciones de formato son ayuda de interfaz. No hay datos de muestra: si falta un contrato del servidor, la pantalla muestra un error real. Existe un único cliente Supabase con clave anon, sin `service_role` en el dispositivo, y los tokens visuales del ADR-007 se usan sin modificar.

5 Monitoreo del plan de trabajo

De las 24 actividades del cronograma, 7 están completadas, 1 ajustada, 2 en curso, 3 con retraso y 8 no iniciadas; las actividades 12, 14 y 18 quedan sin estado hasta que el equipo lo confirme. Los retrasos se concentran en la captura de datos en el dispositivo, que condiciona el hito H5 (MVP integrado e instalable, 19 de octubre).

Id	Actividad	Plan	Estado	Observación
2	Definición de objetivos, alcance y fundamentación	13-ago a 18-ago	**Completado**	Documentación del proyecto entregada
3	Levantamiento y priorización de requisitos (MoSCoW)	17-ago a 21-ago	**Completado**	30 requisitos funcionales y 10 no funcionales; H1 cumplido
4	Diseño de arquitectura y vistas 4+1	20-ago a 31-ago	**Completado**	H2 cumplido
5	Modelo de datos PostgreSQL y migraciones	20-ago a 31-ago	**Completado**	Migraciones 001 a 015 desplegadas
6	Diseño de prototipos de interfaz (wireframes)	24-ago a 2-sep	**Ajustada**	Alcance ampliado a 68 pantallas en nueve iteraciones
10	Despliegue en Supabase, seguridad RLS y CI/CD	1-sep a 11-sep	**Completado**	Supabase y RLS verificados.
11	Desarrollo del motor matemático v1	3-sep a 22-sep	**Completado**	H3 cumplido; 266 pruebas automatizadas
12	Investigación primaria con usuarios	7-sep a 18-sep	**Completado**	Ninguna observación.
13	Sincronización local-first (servidor y cliente)	14-sep a 25-sep	**Completado**	Cliente pendiente: registro de series y almacenamiento local; H4 parcial

Id	Actividad	Plan	Estado	Observación
14	Diseño del plan de backtesting y métricas	21-sep a 30-sep	Completado	Ninguna Observación.
18	Marco legal y tratamiento de datos personales	28-sep a 9-oct	Completado	Términos y Política de Privacidad sin texto en la aplicación.
19	Empaquetado del motor como función de borde	1-oct a 9-oct	Completado	Desplegada y consumida por el cliente
20	Integración frontend-backend y construcción del MVP	1-oct a 13-oct	Con retraso	Integración parcial verificada el 14-oct; faltan registro de series, voz y operación sin conexión
21	Reconocimiento por voz y operación offline	8-oct a 19-oct	Con retraso	Plugin de voz pendiente de prueba en dispositivo físico (tarea C4)
22	Pruebas automatizadas y backtesting	15-oct a 26-oct	En curso	Suites del motor, RLS y cliente en operación; backtesting por completar
23	Informe de avance y evidencias intermedias	21-oct a 30-oct	En curso	Este documento; se elabora antes de lo previsto en el cronograma (H7, 30-oct)
24	Corrección de defectos y endurecimiento	27-oct a 6-nov	No iniciado	Depende de la actividad 22
25	Backend de la pasarela: webhooks y conciliación	2-nov a 13-nov	No iniciado	Requiere el ADR de pagos
26	Curación final del catálogo y datos de producción	3-nov a 12-nov	No iniciado	—
27	Flujo de suscripción y control de acceso en la app	10-nov a 20-nov	No iniciado	Requiere el ADR de pagos
28	Beta cerrada con usuarios reales	10-nov a 20-nov	No iniciado	Depende de la actividad 24
29	Preparación de publicación en Google Play	17-nov a 25-nov	No iniciado	Depende de la actividad 21
30	Consolidación del informe final	12-nov a 26-nov	No iniciado	Depende de la actividad 23
31	Preparación de la defensa y demo funcional	23-nov a 27-nov	No iniciado	Depende de la actividad 28

5.1 Factores que facilitaron el desarrollo

- El backend estuvo completo antes de construir el cliente, de modo que la primera integración ya consumió datos reales.
- Las decisiones se registraron como ADR antes de implementar, lo que permitió cambiar de rumbo (identidad visual y base tecnológica) con trazabilidad.
- Los prototipos en HTML y CSS comparten los tokens del cliente, por lo que su traspaso a Preact es casi directo.
- Los contratos en TypeScript se comparten entre el motor y el cliente.
- Las suites de RLS y del motor actúan como red de seguridad ante cambios.

5.2 Factores que dificultaron el desarrollo

- El plugin de reconocimiento de voz de la comunidad no es compatible con Capacitor 8.
- Faltan contratos del servidor: la regla de la sesión de hoy, el historial de e1RM para las tendencias y la reutilización de planes vigentes (hoy cada `GET /plan` inserta una fila en `sealed_plan`).
- El cambio de identidad visual y de base tecnológica a mitad de la integración obligó a rehacer el esqueleto inicial.
- El perfil de usuario y las sesiones se crean aún a mano, porque no existen las pantallas de perfil inicial y de creación de sesiones.
- El servidor SMTP por defecto de Supabase limita el envío de correos y no sirve para pruebas con varias personas.

5.3 Plan de acción

Retraso o actividad pendiente	Acción	Responsable (según rol)	Fecha objetivo
Integración y MVP (actividad 20)	Registrar series con `POST /sets`, guardarlas primero en el teléfono y completar el flujo de sesión	Bastián (frontend), Fernando (contrato del servidor)	19-oct (H5)
Voz y operación sin conexión (actividad 21)	Probar el fork `@capgo` en dispositivo físico y, si falla, construir un plugin propio en Kotlin; guardar series y plan en almacenamiento local	Bastián	19-oct (H5)
Contratos pendientes del servidor	Regla de la sesión de hoy, vista de historial de e1RM y reutilización del plan vigente	Fernando	26-oct (H6)
Migraciones y correo	Incorporar `v_diagnostico_ultimo` a una migración; configurar SMTP propio con plantillas que incluyan `{{ .Token }}`; probar el alta contra un proyecto real	Favio	26-oct (H6)
Pruebas y backtesting (actividad 22)	Completar la suite automatizada y ejecutar el backtesting con las métricas definidas	Favio	26-oct (H6)
Marco legal (actividad 18)	Redactar y enlazar Términos y Política de Privacidad en la aplicación	Favio / Bastián	
Pasarela y suscripción (actividades 25 y 27)	Redactar el ADR de pagos e inicio de sesión con Google antes de programar	Fernando	Antes del 2-nov
Evidencia de aportes en Git	Confirmar que cada integrante registra sus cambios con su propia identidad en el repositorio	Todo el equipo	Continuo

Referencias

Almonacid, F. (2026a). *ADR-007: Identidad visual de la aplicación móvil* [Registro de decisión de arquitectura]. Repositorio del proyecto RID.

Almonacid, F. (2026b). *ADR-008: Stack de la aplicación móvil* [Registro de decisión de arquitectura]. Repositorio del proyecto RID.

Duoc UC. (2026). *Evaluación Fase 2: Informe avance Proyecto APT* (PTY4614 Capstone) [Guía del estudiante]. Subdirección de Evaluación de Resultados de Aprendizaje; Subdirección de Diseño Instruccional.

Equipo RID. (2026a). *ADR-009: Cuentas, verificación y sesión en la aplicación móvil* [Registro de decisión de arquitectura en estado de propuesta]. Repositorio del proyecto RID.

Equipo RID. (2026b). *Cronograma del proyecto RID — RepInDepth* [Archivo de Microsoft Project, estado al 11 de septiembre de 2026].

Equipo RID. (2026c). *Mapa del código — RID app (iteración 4)* [Documentación técnica]. Repositorio del proyecto RID.

Equipo RID. (2026d). *RID — Inventario de pantallas (diseño, iteraciones 1–9)* [Documentación de diseño]. Repositorio del proyecto RID.

Anexo de evidencias

Evidencias y su ubicación en el repositorio

Evidencia		Se relaciona con
Identidad visual (ADR-007)	`adr/ADR-007-identidad-visual-app-movil.md`	Ajustes; competencia 2
Tokens de diseño	`design/tokens.css`	Competencia 4
Inventario de 68 pantallas	`design/inventario-pantallas.md`	Ajustes; competencia 4
Stack de la aplicación móvil (ADR-008)	`adr/ADR-008-stack-aplicacion-movil.md`	Ajustes; competencia 2
Cuentas, verificación y sesión (ADR-009, propuesta)	`adr/ADR-009-autenticacion-y-sesion.md`	Ajustes; competencia 4
Mapa del código del cliente	`code/mapa-del-codigo.md`	Competencia 4
Contrato del plan sellado del motor	`engine/src/sealed-plan.ts`	Competencia 4
Migraciones del esquema	`supabase/migrations/`	Competencia 3
Código del cliente móvil	`src/` (lib, data, state, components, screens)	Competencia 4
Suite de pruebas RLS		Competencia 1
Pruebas del motor		Competencia 1
Cronograma de Microsoft Project		Competencia 2